

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Простые алгоритмы поиска»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

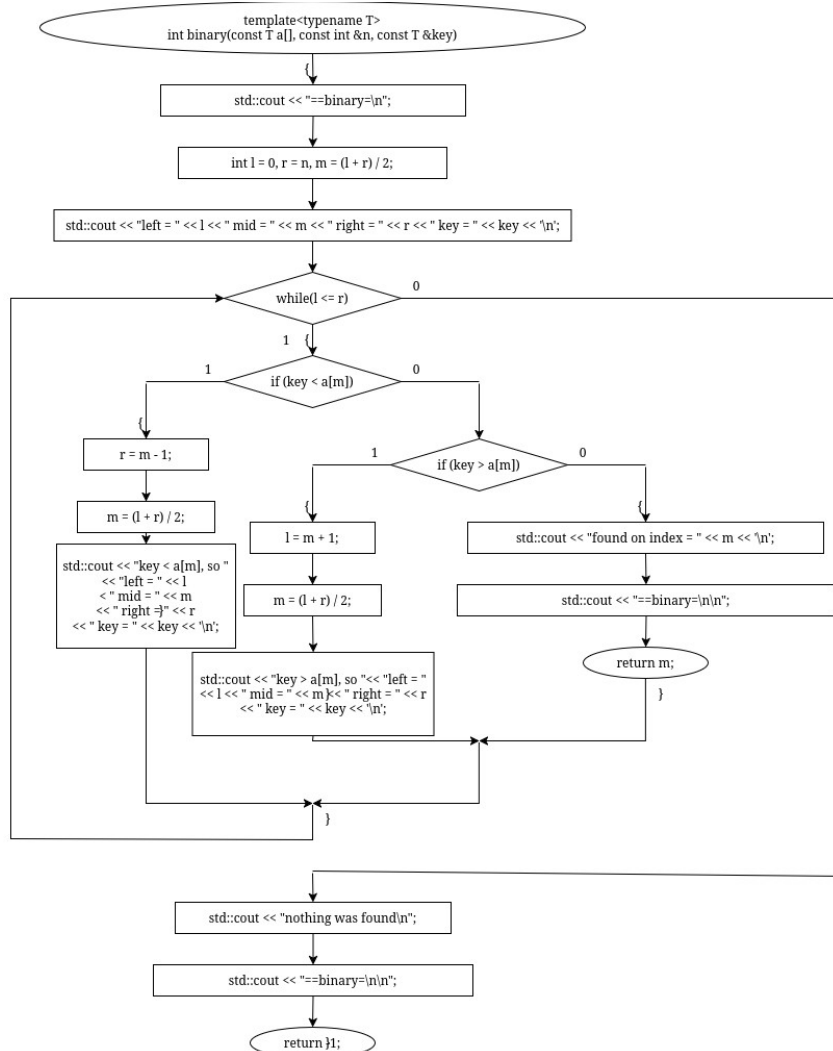
Реализовать алгоритмы поиска:

1. Линейный поиск.
2. Бинарный поиск.
3. Интерполяционный поиск.

АНАЛИЗ АЛГОРИТМА ЛИНЕЙНОГО ПОИСКА

1. Последовательно перебираем каждый элемент массива циклом и сравниваем его с искомым. Когда нашлось совпадение — возвращаем текущий индекс.

БЛОК-СХЕМА



КОД

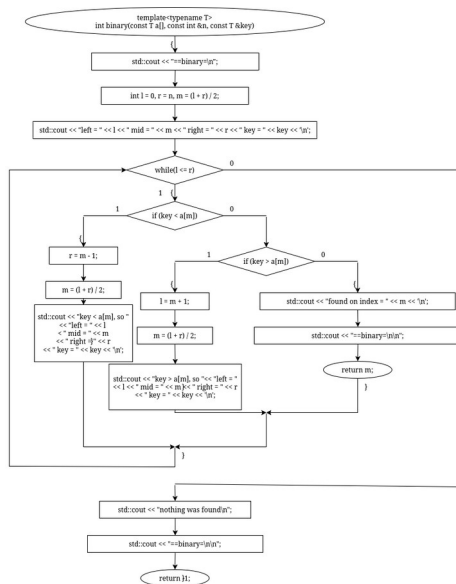
```
#include <iostream>

template<typename T>
int linear(const T a[], const int &n, const T &key) {
    std::cout << "=====linear
search=====\\n";
    for (int i = 0; i < n; i++) {
        std::cout << "idx = " << i << " a[idx] = " << a[i] << " key = " << key << "\\n";
        if (a[i] == key) {
            std::cout << "found on index = " << i << "\\n";
            std::cout << "=====linear
search=====\\n\\n";
            return i;
        }
    }
    std::cout << "nothing was found\\n";
    std::cout << "=====linear
search=====\\n\\n";
    return -1;
}
```

АНАЛИЗ АЛГОРИТМА БИНАРНОГО ПОИСКА

1. Находим границы множества и его середину.
2. Сравниваем ключ с серединным элементом:
 - 2.1 Если искомое значение элемента меньше серединного, то продолжается поиск в левой части множества
 - 2.2 Если искомое значение элемента больше серединного, то продолжается поиск в правой части множества

БЛОК-СХЕМА



КОД

```
#include <iostream>

template<typename T>
int binary(const T a[], const int &n, const T &key) {
    std::cout <<

    "=====binary=====
    =====\n";

    int l = 0, r = n - 1, m = (l + r) / 2;
    std::cout << "left = " << l << " mid = " << m << " right = " << r << " key = " << key << '\n';

    while (l <= r) {
        if (key < a[m]) {
            r = m - 1;
            m = (l + r) / 2;
            std::cout << "key < a[m], so " << "left = " << l << " mid = " << m << " right = " << r << " key = "
            << key <<
                '\n';
        } else if (key > a[m]) {
            l = m + 1;
            m = (l + r) / 2;
            std::cout << "key > a[m], so " << "left = " << l << " mid = " << m << " right = " << r << " key = "
            << key <<
                '\n';
        } else {
            std::cout << "found on index = " << m << '\n';
            std::cout <<

            "=====binary=====
            =====\n\n";

            return m;
        }
    }

    std::cout << "nothing was found\n";
    std::cout <<

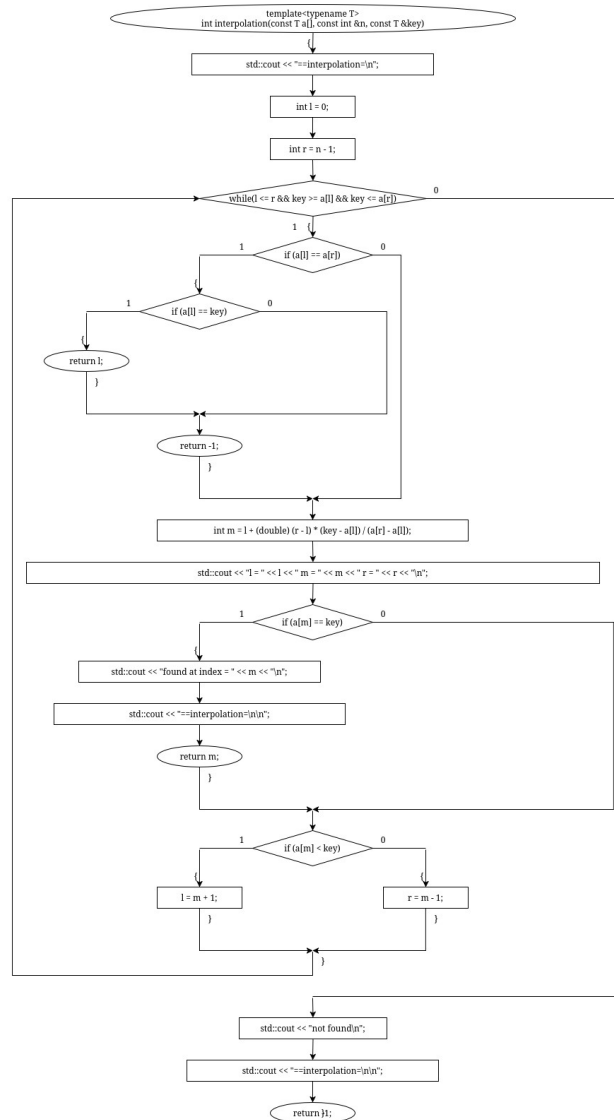
    "=====binary=====
    =====\n\n";

    return -1;
}
```

АНАЛИЗ АЛГОРИТМА ИНТЕРПОЛЯЦИОННОГО ПОИСКА

1. Находим границы множества.
2. Вычисляем середину по формуле $l + (r - l) * (key - a[l]) / (a[r] - a[l])$.
2. Сравниваем ключ с серединным элементом:
 - 2.1 Если искомое значение элемента меньше серединного, то продолжается поиск в левой части множества
 - 2.2 Если искомое значение элемента больше серединного, то продолжается поиск в правой части множества

БЛОК-СХЕМА



Код

```
#include <iostream>
template<typename T>
int interpolation(const T a[], const int &n, const T &key) {
    std::cout <<

    "=====interpolation=====
    =====\n";

    int l = 0;
    int r = n - 1;

    while (l <= r && key >= a[l] && key <= a[r]) {
        if (a[l] == a[r]) {
            if (a[l] == key) return l;
            return -1;
        }

        int m = l + (double) (r - l) * (key - a[l]) / (a[r] - a[l]);

        std::cout << "l = " << l << " m = " << m << " r = " << r << "\n";

        if (a[m] == key) {
            std::cout << "found at index = " << m << "\n";
            std::cout <<

            "=====interpolation=====
            =====\n\n";

            return m;
        }

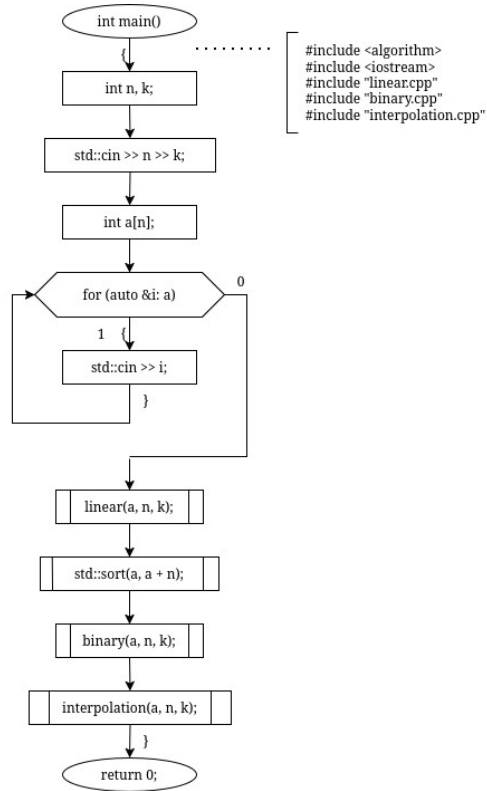
        if (a[m] < key)
            l = m + 1;
        else
            r = m - 1;
    }

    std::cout << "not found\n";
    std::cout <<

    "=====interpolation=====
    =====\n\n";

    return -1;
}
```

БЛОК-СХЕМА ФУНКЦИИ int main()



КОД

```

#include <algorithm>
#include <iostream>
#include "linear.cpp"
#include "binary.cpp"
#include "interpolation.cpp"

```

```

int main() {
    int n, k;
    std::cin >> n >> k;
    int a[n];
    for (auto &i: a) std::cin >> i;
    linear(a, n, k);
    std::sort(a, a + n);
    binary(a, n, k);
    interpolation(a, n, k);
}

```

РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

100 293

2 5 8 11 14 17 20 23 26 29

32 35 38 41 44 47 50 53 56 59

62 65 68 71 74 77 80 83 86 89

92 95 98 101 104 107 110 113 116 119

122 125 128 131 134 137 140 143 146 149

152 155 158 161 164 167 170 173 176 179

182 185 188 191 194 197 200 203 206 209

212 215 218 221 224 227 230 233 236 239

242 245 248 251 254 257 260 263 266 269

272 275 278 281 284 287 290 293 296 299

=====linear
search=====

idx = 0 a[idx] = 2 key = 293

idx = 1 a[idx] = 5 key = 293

idx = 2 a[idx] = 8 key = 293

idx = 3 a[idx] = 11 key = 293

idx = 4 a[idx] = 14 key = 293

idx = 5 a[idx] = 17 key = 293

idx = 6 a[idx] = 20 key = 293

idx = 7 a[idx] = 23 key = 293

idx = 8 a[idx] = 26 key = 293

idx = 9 a[idx] = 29 key = 293

idx = 10 a[idx] = 32 key = 293

idx = 11 a[idx] = 35 key = 293

idx = 12 a[idx] = 38 key = 293

idx = 13 a[idx] = 41 key = 293

idx = 14 a[idx] = 44 key = 293

idx = 15 a[idx] = 47 key = 293
idx = 16 a[idx] = 50 key = 293
idx = 17 a[idx] = 53 key = 293
idx = 18 a[idx] = 56 key = 293
idx = 19 a[idx] = 59 key = 293
idx = 20 a[idx] = 62 key = 293
idx = 21 a[idx] = 65 key = 293
idx = 22 a[idx] = 68 key = 293
idx = 23 a[idx] = 71 key = 293
idx = 24 a[idx] = 74 key = 293
idx = 25 a[idx] = 77 key = 293
idx = 26 a[idx] = 80 key = 293
idx = 27 a[idx] = 83 key = 293
idx = 28 a[idx] = 86 key = 293
idx = 29 a[idx] = 89 key = 293
idx = 30 a[idx] = 92 key = 293
idx = 31 a[idx] = 95 key = 293
idx = 32 a[idx] = 98 key = 293
idx = 33 a[idx] = 101 key = 293
idx = 34 a[idx] = 104 key = 293
idx = 35 a[idx] = 107 key = 293
idx = 36 a[idx] = 110 key = 293
idx = 37 a[idx] = 113 key = 293
idx = 38 a[idx] = 116 key = 293
idx = 39 a[idx] = 119 key = 293
idx = 40 a[idx] = 122 key = 293
idx = 41 a[idx] = 125 key = 293
idx = 42 a[idx] = 128 key = 293
idx = 43 a[idx] = 131 key = 293

idx = 44 a[idx] = 134 key = 293
idx = 45 a[idx] = 137 key = 293
idx = 46 a[idx] = 140 key = 293
idx = 47 a[idx] = 143 key = 293
idx = 48 a[idx] = 146 key = 293
idx = 49 a[idx] = 149 key = 293
idx = 50 a[idx] = 152 key = 293
idx = 51 a[idx] = 155 key = 293
idx = 52 a[idx] = 158 key = 293
idx = 53 a[idx] = 161 key = 293
idx = 54 a[idx] = 164 key = 293
idx = 55 a[idx] = 167 key = 293
idx = 56 a[idx] = 170 key = 293
idx = 57 a[idx] = 173 key = 293
idx = 58 a[idx] = 176 key = 293
idx = 59 a[idx] = 179 key = 293
idx = 60 a[idx] = 182 key = 293
idx = 61 a[idx] = 185 key = 293
idx = 62 a[idx] = 188 key = 293
idx = 63 a[idx] = 191 key = 293
idx = 64 a[idx] = 194 key = 293
idx = 65 a[idx] = 197 key = 293
idx = 66 a[idx] = 200 key = 293
idx = 67 a[idx] = 203 key = 293
idx = 68 a[idx] = 206 key = 293
idx = 69 a[idx] = 209 key = 293
idx = 70 a[idx] = 212 key = 293
idx = 71 a[idx] = 215 key = 293
idx = 72 a[idx] = 218 key = 293

idx = 73 a[idx] = 221 key = 293
idx = 74 a[idx] = 224 key = 293
idx = 75 a[idx] = 227 key = 293
idx = 76 a[idx] = 230 key = 293
idx = 77 a[idx] = 233 key = 293
idx = 78 a[idx] = 236 key = 293
idx = 79 a[idx] = 239 key = 293
idx = 80 a[idx] = 242 key = 293
idx = 81 a[idx] = 245 key = 293
idx = 82 a[idx] = 248 key = 293
idx = 83 a[idx] = 251 key = 293
idx = 84 a[idx] = 254 key = 293
idx = 85 a[idx] = 257 key = 293
idx = 86 a[idx] = 260 key = 293
idx = 87 a[idx] = 263 key = 293
idx = 88 a[idx] = 266 key = 293
idx = 89 a[idx] = 269 key = 293
idx = 90 a[idx] = 272 key = 293
idx = 91 a[idx] = 275 key = 293
idx = 92 a[idx] = 278 key = 293
idx = 93 a[idx] = 281 key = 293
idx = 94 a[idx] = 284 key = 293
idx = 95 a[idx] = 287 key = 293
idx = 96 a[idx] = 290 key = 293
idx = 97 a[idx] = 293 key = 293
found on index = 97

=====linear
search=====

=====binary=====

left = 0 mid = 50 right = 100 key = 293

key > a[m], so left = 51 mid = 75 right = 100 key = 293

key > a[m], so left = 76 mid = 88 right = 100 key = 293

key > a[m], so left = 89 mid = 94 right = 100 key = 293

key > a[m], so left = 95 mid = 97 right = 100 key = 293

found on index = 97

=====binary=====

=====interpolation=====

l = 0 m = 97 r = 99

found at index = 97

=====interpolation=====